

Name - Saikat Sheet
Batch - 25SUB5936_Linux System Programming
ID - 68500

Task 13:

UDP Socket Programming in C

- Create a basic UDP program in C:
- Implement both client and server functionalities within the program.
- The client sends a message "Hello from Client" to the server.
- The server receives the message, prints it, and responds with "Hello from Server."
- The client receives the server's response and displays it.

Code:

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
```

```
#define PORT 8080
#define MAXLINE 1024
```

```
int main() {
    int sockfd;
    char buffer[MAXLINE];
```

```

struct sockaddr_in servaddr, cliaddr;
socklen_t len;

// Create UDP socket
if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
    perror("socket creation failed");
    exit(1);
}

memset(&servaddr, 0, sizeof(servaddr));
memset(&cliaddr, 0, sizeof(cliaddr));

servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = INADDR_ANY;
servaddr.sin_port = htons(PORT);

if (bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0) {
    perror("bind failed");
    close(sockfd);
    exit(1);
}

printf("Server listening on port %d...\n", PORT);

len = sizeof(cliaddr);
int n = recvfrom(sockfd, (char *)buffer, MAXLINE,
                MSG_WAITALL, (struct sockaddr *)&cliaddr, &len);
buffer[n] = '\0';
printf("Client message: %s\n", buffer);

const char *reply = "Hello from Server";
sendto(sockfd, reply, strlen(reply), 0,
        (const struct sockaddr *)&cliaddr, len);
printf("Reply sent.\n");

```

```
    close(sockfd);  
    return 0;  
}
```

client.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/socket.h>  
#include <arpa/inet.h>  
#include <netinet/in.h>
```

```
#define PORT 8080  
#define MAXLINE 1024
```

```
int main() {  
    int sockfd;  
    char buffer[MAXLINE];  
    struct sockaddr_in servaddr;  
  
    // Create UDP socket  
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {  
        perror("socket creation failed");  
        exit(1);  
    }  
  
    memset(&servaddr, 0, sizeof(servaddr));  
    servaddr.sin_family = AF_INET;  
    servaddr.sin_port = htons(PORT);  
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1"); // localhost  
  
    const char *msg = "Hello from Client";
```

```

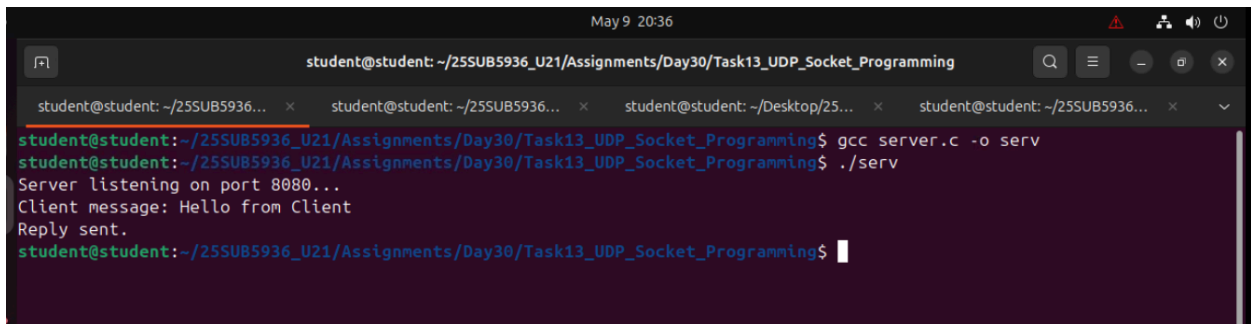
sendto(sockfd, (const char *)msg, strlen(msg), 0,
        (const struct sockaddr *)&servaddr, sizeof(servaddr));
printf("Message sent to server.\n");

int n = recvfrom(sockfd, (char *)buffer, MAXLINE,
                 MSG_WAITALL, NULL, NULL);
buffer[n] = '\0';
printf("Server reply: %s\n", buffer);

close(sockfd);
return 0;
}

```

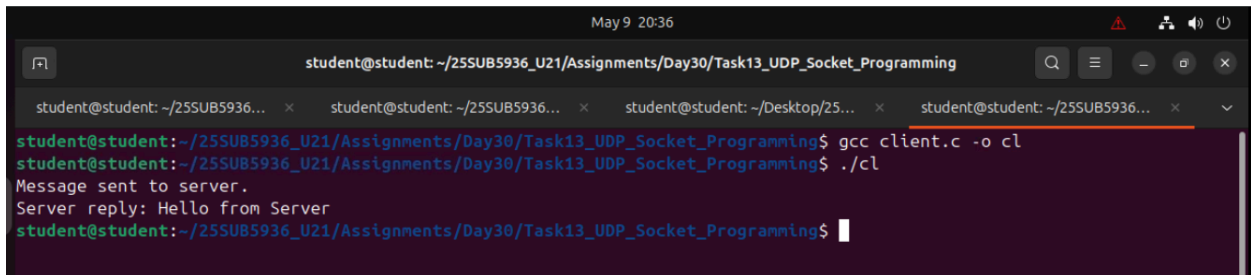
Output:



```

May 9 20:36
student@student: ~/25SUB5936_U21/Assignments/Day30/Task13_UDP_Socket_Programming
student@student: ~/25SUB5936_U21/Assignments/Day30/Task13_UDP_Socket_Programming$ gcc server.c -o serv
student@student: ~/25SUB5936_U21/Assignments/Day30/Task13_UDP_Socket_Programming$ ./serv
Server listening on port 8080...
Client message: Hello from Client
Reply sent.
student@student: ~/25SUB5936_U21/Assignments/Day30/Task13_UDP_Socket_Programming$

```



```

May 9 20:36
student@student: ~/25SUB5936_U21/Assignments/Day30/Task13_UDP_Socket_Programming
student@student: ~/25SUB5936_U21/Assignments/Day30/Task13_UDP_Socket_Programming$ gcc client.c -o cl
student@student: ~/25SUB5936_U21/Assignments/Day30/Task13_UDP_Socket_Programming$ ./cl
Message sent to server.
Server reply: Hello from Server
student@student: ~/25SUB5936_U21/Assignments/Day30/Task13_UDP_Socket_Programming$

```